



Homework #2

TurtleBot3 Digit Formation in Gazebo

Capstone Design [202601-ICE/CSE4020]

Prof.: Dr. Oybek Eraliyev

Individual Assignment

1. Overview

In this homework, you will work with **multiple TurtleBot3 robots in a Gazebo simulation**. Your task is to spawn **TurtleBot3 Burger robots** in an empty world and command each one to move to a specific target position such that, when viewed from above, the robots' final positions form the **last 3 digits of your student ID**.

Think of it like a dot-matrix display: each robot is a “dot,” and by placing enough dots at the right coordinates, you can form recognizable digits. For example, if your student ID ends in 164, you need to arrange your robots so that their final positions clearly spell out **1, 6, and 4** when seen from a top-down perspective.

All robots must begin in a **horizontal line at the bottom of the Gazebo world** and then move to their target positions when you run your code. Please consider 15 robots as the required minimum. If you want higher precision and cleaner digits, you can deal with more.

This assignment builds on your HW1 experience with ROS topics, publishers, and services, and introduces you to Gazebo simulation, multi-robot systems, and launch files.

2. Getting Started

This assignment requires you to work with several tools and concepts that you may not have used before. Rather than walking through every detail here, this section points you to the key things you need to set up and learn, along with references to help you.

2.1 Prerequisites

Before starting, make sure you have the following ready:

- ROS Noetic installed and sourced on Ubuntu 20.04.
- TurtleBot3 packages installed ([ros-noetic-turtlebot3](#), [ros-noetic-turtlebot3-simulations](#), [ros-noetic-turtlebot3-msgs](#)) and `TURTLEBOT3_MODEL` set to `burger`.
- A catkin workspace created and sourced. If you have not done this before, look up how to create one — it takes under 5 minutes.

- Gazebo tested with a single TurtleBot3. Run `roslaunch turtlebot3_gazebo turtlebot3_empty_world.launch` and confirm you see a robot on an empty ground plane.

2.2 Key Concepts to Learn

You will need to understand the following concepts to complete this assignment. Research them using the references provided below:

- **ROS Launch Files** — XML files that start multiple nodes and configure parameters in one command. You will write a launch file that spawns Gazebo and multiple robots.
- **Namespaces** — The mechanism that gives each robot its own set of topics (e.g., `/tb3_1/cmd_vel`, `/tb3_2/cmd_vel`). Without namespaces, all robots would share the same topics and you could not control them independently.
- **Gazebo Model Spawning** — The `spawn_model` node from the `gazebo_ros` package places a robot model into the simulation at a specified position.
- **Odometry** — Each robot publishes its position and orientation on its `/tb3_x/odom` topic. You will use this to know where each robot is and steer it toward its target.

2.3 Useful References

The following resources will help you throughout this assignment:

- **TurtleBot3 e-Manual** — <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/>
- **ROS Wiki: roslaunch** — <http://wiki.ros.org/roslaunch/XML>
- **ROS Wiki: Gazebo spawn_model** — http://wiki.ros.org/gazebo_ros
- **ROS Wiki: Namespaces** — <http://wiki.ros.org/Names>
- **ROS Wiki: catkin workspaces** — http://wiki.ros.org/catkin/Tutorials/create_a_workspace

3. Multi-Robot Spawning

The core technical challenge of this assignment is spawning multiple TurtleBot3 robots in a single Gazebo world, each under its own namespace. This section gives you the essential pattern — you are expected to extend it for your full robot set.

3.1 The Spawning Pattern

Your launch file needs to do two things: start Gazebo with an empty world, then spawn each robot under a unique namespace at a unique position. Here is the pattern for spawning one robot:

```
<launch>
  <!-- Start Gazebo -->
  <include file="$(find gazebo_ros)/launch/empty_world.launch">
    <arg name="paused" value="false" />
    <arg name="gui" value="true" />
  </include>

  <!-- Robot 1 -->
  <group ns="tb3_1">
    <param name="robot_description"
      command="$(find xacro)/xacro $(find turtlebot3_description)
        /urdf/turtlebot3_burger.urdf.xacro" />

    <node pkg="gazebo_ros" type="spawn_model"
      name="spawn_tb3_1"
      args="-urdf -model tb3_1
        -x 0.0 -y -4.0 -z 0.0
        -param robot_description" />
  </group>

  <!-- Robot 2: same pattern, different ns, model name, and x -->
  <!-- ... -->

</launch>
```

The `<group ns="tb3_1">` tag creates a namespace. Everything inside gets its topics prefixed with `/tb3_1/` — so this robot's velocity topic is `/tb3_1/cmd_vel` and its odometry is `/tb3_1/odom`. To add more robots, repeat the `<group>` block with a new namespace (`tb3_2`, `tb3_3`, etc.), a unique `-model` name, and a different `-x` position so they don't overlap in the starting line.

Starting positions: All robots must start in a horizontal line at the bottom of the world. Keep the `y` coordinate the same for all robots (e.g., `-4.0`) and space them along the `x` axis. The exact spacing is your decision.

4. Performance Optimization

Spawning 15+ robots can be demanding. Since this assignment uses an empty world with no obstacles, you can safely disable components that consume significant resources.

4.1 Remove the LiDAR Sensor

Each TurtleBot3 Burger runs a 360° LiDAR sensor that performs hundreds of raycasts per cycle. With 15+ robots, this creates thousands of unnecessary calculations. Since there are no obstacles to detect, you can remove the LiDAR from the robot model.

To do this, copy the TurtleBot3 URDF files into your package and edit the `.urdf.xacro` file to comment out or remove the LiDAR-related `<joint>`, `<link>`, and `<gazebo>` plugin sections.

Then update your launch file to point to your modified model. This alone can reduce CPU usage by 50% or more.

Why is this safe? You will avoid collisions not through sensing, but through careful path design — by staggering robot start times, grouping movements, or choosing non-crossing paths. This is good engineering practice: preventing problems through planning rather than reactive sensing.

4.2 Reduce Visual Load

Inside the Gazebo window, disable **Shadows** through the Scene settings. This reduces GPU load while keeping all robots and their movement fully visible. The simulation and physics are completely unaffected.

Hints

- Start small. Get 2–3 robots spawning and moving before scaling up to your full set.
- Use rostopic list after launching to verify each robot has its own namespaced topics.
- Plan your digit coordinates on paper or a spreadsheet before writing any code. Map out (x, y) targets for every robot.
- To move a robot to a target, subscribe to its /odom topic to track its position, then publish Twist messages to steer it. A simple proportional controller works well.
- Use Gazebo's top-down view (View menu) to verify your digits are readable from above.

5. Task Description

Your task, step by step, is as follows.

1. **Create a ROS package and launch file** that starts Gazebo with an empty world and spawns at least 15 TurtleBot3 Burger robots. All robots must start in a **horizontal line at the bottom** of the Gazebo world. Each robot must have a unique namespace.
2. **Design your digit layout.** Determine the target (x, y) positions for each robot so that their final arrangement forms the last 3 digits of your student ID. Each digit should occupy its own region of the world so they are visually separated. Sketch your layout on paper first.
3. **Write a Python ROS node** that commands each robot to move from its starting position to its assigned target position. Your script should publish to each robot's namespaced `cmd_vel` topic and subscribe to each robot's `odom` topic to track progress. No manual teleoperation is allowed.
4. **Avoid collisions by design.** Since robots start in a line and move to scattered positions, their paths could cross. Plan your movement strategy to prevent this — for example, stagger start times, move robots in groups, or choose non-crossing paths.
5. **Verify and record.** Once all robots have reached their positions, switch to Gazebo's top-down view and confirm that the 3 digits are clearly readable. Record the full process — from the starting line through all movements to the final formation.

6. Grading Rubric

Your submission will be evaluated according to the following criteria. The total score is 100 points.

Criterion	Points	Details
Digit readability <i>top-down view</i>	30	All 3 digits are clearly recognizable from a top-down perspective, well-proportioned, visually separated, and match the last 3 digits of the student ID.
Robot movement and control <i>autonomous navigation to targets</i>	30	All robots navigate from their starting positions to their target positions using velocity commands and odometry feedback. Movement is smooth, collision-free, and fully autonomous.
Screen recording clearance <i>full process must be visible</i>	15	Recording clearly shows the entire process: starting formation (horizontal line), all robots in motion, and the final digit formation from a top-down view. No cuts or edits that skip the movement phase.
Multi-robot launch file <i>correct namespacing</i>	15	At least 15 robots spawn correctly with unique namespaces. Each robot has its own topics and can be controlled independently.
Starting formation <i>horizontal line at the bottom</i>	10	All robots begin in a horizontal line at the bottom of the Gazebo world before moving to their target positions.

7. Submission Instructions

You must submit the following items through the specified platforms.

1. **Source code and launch files** — Submit your complete ROS package (including the launch file, any modified URDF files, and your Python node) to the **GitHub organization of our course – IUT-SCS**. The package should be buildable with `catkin_make`. Name your package `hw2_STUDENTID` (for example, `hw2_U2010164`). Further details are only available through the **Discord server**.

2. **Screen recording** — A video showing the full process: Gazebo launching with all robots in the starting line, the robots moving to their target positions, and the final digit formation viewed from above. Keep the file size low. Upload this file to **e-Class**.

Late submissions are not allowed!

8. Important Notes

1. This is an **individual assignment**. You must write your own code independently. Sharing code with classmates is considered academic dishonesty.
2. If your laptop struggles with many robots, apply the performance optimizations described in Sec 4. Removing the LiDAR sensor alone makes a significant difference.
3. Plan your digit coordinates on paper or in a spreadsheet **before** writing any code. This will save you significant trial and error.
4. Test incrementally. Start with 2–3 robots, get them spawning and moving correctly, then scale up. Do not try to spawn 15+ robots on your first attempt.
5. If you encounter issues, refer to the resources listed in Section 2.3. The ROS Wiki and TurtleBot3 e-Manual are your primary references.
6. Start early. Setting up multi-robot spawning, designing digit layouts, and writing movement controllers takes more time than you might expect.

Good luck! If you have any questions, reach out to the Education Assistant of this course.